

Time Complexity of Erlang Programs

Christian Rinderknecht

16th August 2026

Let us note e any **Erlang expression**, i.e., a function call or a constant, that can be rewritten by **Erlang** clauses. This process is called *computation* and (1) may never end (*loop*), (2) ends with a type error (as $1 + []$), (3) ends with a match error (an expression containing a function call which matches no head), (4) ends with a value (an expression which is a constant).

This procedure can lead us to define the *cost* (or *complexity*) of computing an **Erlang** expression as the number of rewrite steps until reaching a value. Accordingly, a loop has an infinite cost and a value has a cost of zero. We denote the cost of expression e by $\llbracket e \rrbracket$, and the value of an expression e , assuming it exists, by $\mathcal{V}(e)$. Hence, if e is a value, i.e., $e = \mathcal{V}(e)$, then $\llbracket e \rrbracket = \llbracket \mathcal{V}(e) \rrbracket = 0$.

1 Joining two lists

Consider the following **Erlang** module:

```
-module(join).  
-export([join/2]).  
join([],L)    -> L;                               % Clause 1  
join([H|T],L) -> [H|join(T,L)].                     % Clause 2
```

The cost of a function call like $\llbracket \text{join}(e_1, e_2) \rrbracket$ is, by definition, the sum of the costs of rewriting e_1 and e_2 , plus the cost of rewriting the call with the corresponding values $\mathcal{V}(e_1)$ and $\mathcal{V}(e_2)$ as arguments:

$$\llbracket \text{join}(e_1, e_2) \rrbracket \triangleq \llbracket e_1 \rrbracket + \llbracket e_2 \rrbracket + \llbracket \text{join}(\mathcal{V}(e_1), \mathcal{V}(e_2)) \rrbracket$$

We apply this definition to each clause in the definition:

$$\begin{aligned}\llbracket \text{join}([], L) \rrbracket &= 1 \\ \llbracket \text{join}([H|T], L) \rrbracket &= 1 + \llbracket \text{join}(T, L) \rrbracket\end{aligned}$$

The “1” in the previous equations are due to the fact that at least one rewrite step is necessary. Here, all **Erlang** variables, L , H and T , denote values, since their binding takes place after the call. So, $\mathcal{V}(L) = L$ etc. Note that, since there are no rewrite rules for $[H|T]$, the cost of it is 0 and $\mathcal{V}([H|T]) = [H|T]$.

Let us now define the cost in terms of the size of the input, i.e., the arguments. The function call $\text{join}(e_1, e_2)$ has two lists as arguments. The usual way to measure the size of a list is to consider the number of elements it contains, i.e., its length. Here, the size of the input is characterized by a pair (n, m) where $n = \text{len}(e_1)$ and $m = \text{len}(e_2)$. Let us define the cost of $\text{join}/2$ in terms of the input's size as

$$\mathcal{C}_{n,m}^{\text{join}/2} \triangleq \llbracket \text{join}(e_1, e_2) \rrbracket \quad \text{where } \text{len}(e_1) = n \text{ and } \text{len}(e_2) = m$$

Using this definition, the previous equations are equivalent to

$$\begin{aligned} \mathcal{C}_{0,m}^{\text{join}/2} &= 1 \\ \mathcal{C}_{n+1,m}^{\text{join}/2} &= 1 + \mathcal{C}_{n,m}^{\text{join}/2} \quad \text{where } n, m \geq 0 \end{aligned}$$

Notice that the length of the second list, m , does not impact the cost. We need now to express $\mathcal{C}_{n+1,m}^{\text{join}/2}$ in terms of n . The trick consists in making the difference of two successive terms in the series and then summing up these differences:

$$\begin{aligned} \mathcal{C}_{n+1,m}^{\text{join}/2} - \mathcal{C}_{n,m}^{\text{join}/2} &= 1 \quad \text{where } n \geq 0 \\ \sum_{n=0}^p (\mathcal{C}_{n+1,m}^{\text{join}/2} - \mathcal{C}_{n,m}^{\text{join}/2}) &= \sum_{n=0}^p 1 \quad \text{where } p \geq 0 \\ \mathcal{C}_{p+1,m}^{\text{join}/2} - \mathcal{C}_{0,m}^{\text{join}/2} &= p + 1 \end{aligned}$$

Replacing $p + 1$ by n we finally get

$$\begin{aligned} \mathcal{C}_{n,m}^{\text{join}/2} - \mathcal{C}_{0,m}^{\text{join}/2} &= n \quad \text{where } n > 0 \\ \mathcal{C}_{n,m}^{\text{join}/2} &= n + 1 \end{aligned}$$

Since that replacing n by 0 in the latter formula gives

$$\mathcal{C}_{0,m}^{\text{join}/2} = 0 + 1 = 1$$

it is thus possible to have only one formula for the cost:

$$\mathcal{C}_{n,m}^{\text{join}/2} = n + 1 \quad \text{where } n, m \geq 0$$

Let us take an example:

$$\begin{aligned} \text{join}([1,2,3], [4,5]) &\xrightarrow{2} [1 | \text{join}([2,3], [4,5])] \\ &\xrightarrow{2} [1 | [2 | \text{join}([3], [4,5])]] \\ &\xrightarrow{2} [1 | [2 | [3 | \text{join}([], [4,5])]]] \\ &\xrightarrow{1} [1 | [2 | [3 | [4,5]]]] \end{aligned}$$

$$= [1, 2, 3, 4, 5]$$

The formula predicted $\mathcal{C}_{\text{len}([1,2,3]), \text{len}([4,5])}^{\text{join}/2} = \mathcal{C}_{3,2}^{\text{join}/2} = 3 + 1 = 4$ and we indeed had 4 rewrite steps until the result.

Futhermore, it is important to have a clear understanding of the behaviour of the cost when the input becomes sufficiently big. This insight is provided by considering an equivalent function¹ when the variable is arbitrarily big:

$$\mathcal{C}_{n,m}^{\text{join}/2} \sim n \quad \text{where } n \rightarrow +\infty$$

These kinds of asymptotical expressions are also very useful when comparing the cost of two Erlang functions, perhaps two versions of the same function.

The idea behind our concept of cost is that the time spent in computing an expression is proportional to the cost. So, here, it means that the time necessary to compute the joining of two lists is proportional to the length of the first list, when the list grows very long. In particular, doubling the size of the first list will approximatively double the time to get the result, whereas doubling the length of the second list will have no impact.

2 Reversing a list

2.1 A naive version

Consider the following Erlang module:

```
-module(rev).
-export([rev/1]).
join([], L)      -> L;
join([H|T], L)   -> [H|join(T, L)].

rev([])          -> [];
rev([H|T])       -> join(rev(T), [H]).
```

% Clause 3
% Clause 4

The cost of a function call like $\llbracket \text{rev}(e) \rrbracket$ is, by definition, the sum of the costs of rewriting e and the cost of rewriting the call with the corresponding value $\mathcal{V}(e)$ as argument.

$$\llbracket \text{rev}(e) \rrbracket \triangleq \llbracket e \rrbracket + \llbracket \text{rev}(\mathcal{V}(e)) \rrbracket$$

Considering each clause separately, we can therefore write the equations defining the costs

$$\begin{aligned} \llbracket \text{rev}([]) \rrbracket &= 1 \\ \llbracket \text{rev}([H|T]) \rrbracket &= 1 + \llbracket \text{join}(\text{rev}(T), [H]) \rrbracket \\ &= 1 + (\llbracket \text{rev}(T) \rrbracket + \llbracket \text{join}(\mathcal{V}(\text{rev}(T)), [H]) \rrbracket) \end{aligned}$$

¹Two functions $f(x)$ and $g(x)$ are equivalent at $+\infty$ if $\lim_{x \rightarrow +\infty} f(x)/g(x) = 1$.

Let us now define the cost in terms of the size of the input, i.e., the argument. The function call $\text{rev}(e)$ has one list as argument. The usual way to measure the size of a list is to consider the number of elements it contains, i.e., its length. Here, the size of the input is characterized by an integer n where $n = \text{len}(e)$. Let us define the cost of $\text{rev}/1$ in terms of n :

$$\mathcal{C}_n^{\text{rev}/1} \triangleq \llbracket \text{rev}(e) \rrbracket \quad \text{where } \text{len}(e) = n$$

Using this definition, the previous equations are equivalent to

$$\begin{aligned} \mathcal{C}_0^{\text{rev}/1} &= 1 \\ \mathcal{C}_{n+1}^{\text{rev}/1} &= 1 + \mathcal{C}_n^{\text{rev}/1} + \mathcal{C}_{n,1}^{\text{join}/2} && \text{since } \text{len}(\text{rev}(T)) = \text{len}(T) \\ &= 1 + \mathcal{C}_n^{\text{rev}/1} + (n+1) && \text{where } n \geq 0 \\ &= 2 + n + \mathcal{C}_n^{\text{rev}/1} \end{aligned}$$

Replacing n by $n-1$, we get

$$\mathcal{C}_n^{\text{rev}/1} = 1 + n + \mathcal{C}_{n-1}^{\text{rev}/1} \quad \text{where } n > 0$$

We would like rather to express $\mathcal{C}_n^{\text{rev}/1}$ in terms of n only. The trick consists in making the difference of two successive terms in the series and then summing up these differences:

$$\begin{aligned} \mathcal{C}_n^{\text{rev}/1} - \mathcal{C}_{n-1}^{\text{rev}/1} &= 1 + n && \text{where } n > 0 \\ \sum_{n=1}^p (\mathcal{C}_n^{\text{rev}/1} - \mathcal{C}_{n-1}^{\text{rev}/1}) &= \sum_{n=1}^p (1 + n) && \text{where } p > 0 \\ \mathcal{C}_p^{\text{rev}/1} - \mathcal{C}_0^{\text{rev}/1} &= \sum_{n=2}^{p+1} n \\ \mathcal{C}_p^{\text{rev}/1} - 1 &= \sum_{n=2}^{p+1} n \\ \mathcal{C}_p^{\text{rev}/1} &= \sum_{n=1}^{p+1} n \end{aligned}$$

Let us double each side of the equality:

$$2 \times \mathcal{C}_p^{\text{rev}/1} = 2 \times \sum_{n=1}^{p+1} n = \sum_{n=1}^{p+1} n + \sum_{n=1}^{p+1} n$$

Let us change n into $p-n+2$ in the second sum:

$$2 \times \mathcal{C}_p^{\text{rev}/1} = \sum_{n=1}^{p+1} n + \sum_{n=1}^{p+1} (p-n+2)$$

$$\begin{aligned}
&= \sum_{n=1}^{p+1} (n + (p - n + 2)) \\
&= \sum_{n=1}^{p+1} (p + 2) = (p + 1)(p + 2)
\end{aligned}$$

By changing p into n we finally get

$$\mathcal{C}_n^{\text{rev}/1} = (n + 1)(n + 2)/2 \quad \text{where } n > 0$$

Since that replacing n by 0 in the latter formula gives

$$\mathcal{C}_0^{\text{rev}/1} = (0 + 1)(0 + 2)/2 = 1$$

we can therefore gather the two cases into one as

$$\mathcal{C}_n^{\text{rev}/1} = (n + 1)(n + 2)/2 \quad \text{where } n \geq 0$$

Let us take an example:

$$\begin{aligned}
\text{rev}([1, 2, 3]) &\xrightarrow{4} \text{join}(\text{rev}([2, 3]), [1]) \\
&\xrightarrow{4} \text{join}(\text{join}(\text{rev}([3]), [2]), [1]) \\
&\xrightarrow{4} \text{join}(\text{join}(\text{join}(\text{rev}([]), [3]), [2]), [1]) \\
&\xrightarrow{3} \text{join}(\text{join}(\text{join}([], [3]), [2]), [1]) \\
&\xrightarrow{1} \text{join}(\text{join}([3], [2]), [1]) \\
&\xrightarrow{2} \text{join}([3 | \text{join}([], [2])], [1]) \\
&\xrightarrow{1} \text{join}([3 | [2]], [1]) \\
&= \text{join}([3, 2], [1]) \\
&\xrightarrow{2} [3 | \text{join}([2], [1])] \\
&\xrightarrow{2} [3 | [2 | \text{join}([], [1])]] \\
&\xrightarrow{1} [3 | [2 | [1]]] \\
&= [3, 2, 1]
\end{aligned}$$

The formula for the cost stated $\mathcal{C}_{\text{len}([1, 2, 3])}^{\text{rev-join}/2} = \mathcal{C}_3^{\text{rev-join}/2} = (3+1)(3+2)/2 = 10$, and we indeed made 10 rewrites until reaching the value $[3, 2, 1]$.

Asymptotically, we have the equivalence

$$\mathcal{C}_n^{\text{rev}/1} \sim \frac{1}{2}n^2 \quad \text{where } n \rightarrow +\infty$$

The idea behind our concept of cost is that the time spent in computing an expression is proportional to the cost. So, here, it means that the time needed to reverse a list by $\text{rev}/1$ is proportional to half of the square of the list length, when the list is big enough. In particular, multiplying by 10 the size of a big input list will multiply at least by 50 the computing time.

2.2 A better version

Consider the following list reversing using an *accumulator*:

```
-module(rev_bis).
-export([rev_bis/1]).
rev_bis(L)      -> rev_join(L, []).      % Clause 5

rev_join([],A)  -> A;                    % Clause 6
rev_join([H|T],A) -> rev_join(T,[H|A]). % Clause 7
```

The cost of a function call like $\llbracket \text{rev_bis}(e) \rrbracket$ is, by definition, the sum of the costs of rewriting e and the cost of rewriting the call with the corresponding value $\mathcal{V}(e)$ as argument. The same idea applies to $\text{rev_join}(e_1, e_2)$.

$$\begin{aligned}\llbracket \text{rev_bis}(e) \rrbracket &\triangleq \llbracket e \rrbracket + \llbracket \text{rev_bis}(\mathcal{V}(e)) \rrbracket \\ \llbracket \text{rev_join}(e_1, e_2) \rrbracket &\triangleq \llbracket e_1 \rrbracket + \llbracket e_2 \rrbracket + \llbracket \text{rev_join}(\mathcal{V}(e_1), \mathcal{V}(e_2)) \rrbracket\end{aligned}$$

Considering each clause separately, we can therefore write the equations defining the costs

$$\begin{aligned}\llbracket \text{rev_bis}(L) \rrbracket &= 1 + \llbracket \text{rev_join}(L, []) \rrbracket \\ \llbracket \text{rev_join}([], A) \rrbracket &= 1 \\ \llbracket \text{rev_join}([H|T], A) \rrbracket &= 1 + \llbracket \text{rev_join}(T, [H|A]) \rrbracket\end{aligned}$$

Let us now define the cost in terms of the size of the input, i.e., the argument. The function call $\text{rev_bis}(e)$ has one list as argument. The usual way to measure the size of a list is to consider the number of elements it contains, i.e., its length. Here, the size of the input is characterized by an integer n where $n = \text{len}(e)$. The same line of thought applies to $\text{rev_join}/2$. Let us define the cost of $\text{rev_bis}/1$ and $\text{rev_join}/2$ in terms of the input size:

$$\begin{aligned}\mathcal{C}_n^{\text{rev_bis}/1} &\triangleq \llbracket \text{rev_bis}(e) \rrbracket && \text{where } \text{len}(e) = n \\ \mathcal{C}_{n,m}^{\text{rev_join}/2} &\triangleq \llbracket \text{rev_join}(e_1, e_2) \rrbracket && \text{where } \text{len}(e_1) = n \text{ and } \text{len}(e_2) = m\end{aligned}$$

Using these definitions, the previous equations are equivalent to

$$\begin{aligned}\mathcal{C}_n^{\text{rev_bis}/1} &= 1 + \mathcal{C}_{n,0}^{\text{rev_join}/2} \\ \mathcal{C}_{0,m}^{\text{rev_join}/2} &= 1 \\ \mathcal{C}_{n+1,m}^{\text{rev_join}/2} &= 1 + \mathcal{C}_{n,m+1}^{\text{rev_join}/2} && \text{where } n, m \geq 0\end{aligned}$$

We would like rather to express $\mathcal{C}_n^{\text{rev_bis}/1}$, respectively $\mathcal{C}_{n,m}^{\text{rev_join}/2}$, in terms of n only, respectively n and m . The trick consists in making the difference of two successive terms in the series $\{\mathcal{C}_{n,m}^{\text{rev_join}/2}\}_n$ and then summing up these differences:

$$\mathcal{C}_{n,m}^{\text{rev_join}/2} - \mathcal{C}_{n-1,m+1}^{\text{rev_join}/2} = 1$$

$$\begin{aligned}
C_{n-1,m+1}^{\text{rev_join}/2} - C_{n-2,m+2}^{\text{rev_join}/2} &= 1 \\
&\vdots \\
C_{1,m+n-1}^{\text{rev_join}/2} - C_{0,m+n}^{\text{rev_join}/2} &= 1
\end{aligned}$$

$$\begin{aligned}
C_{n,m}^{\text{rev_join}/2} - C_{0,m+n}^{\text{rev_join}/2} &= n \\
C_{n,m}^{\text{rev_join}/2} - 1 &= n \\
C_{n,m}^{\text{rev_join}/2} &= n + 1 \quad \text{where } n > 0
\end{aligned}$$

Since that replacing n by 0 in the last formula leads to

$$C_{0,m}^{\text{rev_join}/2} = 0 + 1 = 1$$

we can gather all the cases into one formula:

$$C_{n,m}^{\text{rev_join}/2} = n + 1 \quad \text{where } n, m \geq 0$$

Therefore

$$C_n^{\text{rev_bis}/1} = n + 2 \quad \text{where } n \geq 0$$

Let us take an example:

$$\begin{aligned}
\text{rev_bis}([1,2,3]) &\xrightarrow{5} \text{rev_join}([1,2,3], []) \\
&\xrightarrow{7} \text{rev_join}([2,3], [1]) \\
&\xrightarrow{7} \text{rev_join}([3], [2,1]) \\
&\xrightarrow{7} \text{rev_join}([], [3,2,1]) \\
&\xrightarrow{6} [3,2,1]
\end{aligned}$$

The formula for the cost stated $C_{\text{len}([3,2,1])}^{\text{rev_bis}/1} = C_3^{\text{rev_bis}/1} = 3 + 2 = 5$ and we indeed made 5 rewrites before reaching the result $[3,2,1]$.

Asymptotically, we have the law

$$C_n^{\text{rev_bis}/1} \sim n \quad \text{where } n \rightarrow +\infty$$

The idea behind our concept of cost is that the time spent in computing an expression is proportional to the cost. So, here, it means that the time needed to reverse a list by `rev_bis/1` is proportional to the length of the list, when the list is long enough. We can also now compare the cost of `rev_bis/1` with `rev/1`:

$$\begin{aligned}
C_n^{\text{rev}/1} &= \frac{n+1}{2} C_n^{\text{rev_bis}/1} \\
C_n^{\text{rev}/1} &\sim \frac{n}{2} C_n^{\text{rev_bis}/1} \quad \text{where } n \rightarrow +\infty
\end{aligned}$$

Clearly, `rev_bis/1` is better than `rev/1`.